

SPM Modular Project: Modulo4

SPM course a.y. 25/26

Partitioned Hash Join with Duplicates

Module 4: Distributed Implementation using MPI

In the previous modules, you studied the partitioned hash join first as a vectorized kernel, then as a shared-memory parallel application using C++ concurrency mechanisms and OpenMP. In this final module, you will study the same algorithm in a distributed-memory setting using MPI.

The goal is not simply to distribute the computation across multiple nodes, but to understand how communication costs interact with local computation, and how the performance of the algorithm changes when memory bandwidth and memory capacity are distributed across the cluster.

The overall algorithm remains unchanged. As in the previous modules, both input relations are partitioned according to the mapping function. Records assigned to the same partition must be brought together so that the corresponding local join can be performed. The final result is obtained by accumulating the contributions produced by all processes.

Sequential Reference Implementation

The MPI implementation must be evaluated against the same sequential baseline used in the previous modules. If you improved the provided sequential implementation in Modules 2 and 3, you must use that same improved sequential version as the baseline in this module as well. The purpose is to ensure a fair comparison between execution models, not between different algorithms.

MPI Implementation

You are required to implement an MPI version of the algorithm. The implementation must preserve the same overall algorithmic structure used in the previous modules. It must produce exactly the same output, using the same mapping function and the same input generation scheme.

The distributed version should reorganize the computation so that each MPI process can perform a local join on the data assigned to it. How this redistribution is achieved is part of the design space of the module.

Optional Hybrid Version: You may optionally provide a hybrid implementation using MPI+OpenMP. If you do so, this version should be discussed separately in the report and compared with the pure MPI version.

Workload and Input Data

Your performance evaluation must include at least one uniform dataset, generated using the same scheme adopted in the previous modules. You may optionally include an additional skewed dataset and discuss its impact on distributed performance.

Correctness

All implementations must produce the same output, namely:

- the total number of matches (join_count)
- the checksum values used for verification

As in the previous modules, you must implement a validation strategy. For small inputs, stronger checks can be performed against a simpler reference implementation. For larger inputs, correctness is verified by comparing the final aggregated output values (join_count, checksum1, and checksum2) across all implementations. The verification mechanism should be kept separate from the measured region.

Performance Evaluation

You must evaluate the performance of your MPI implementation on the nodes of the spmcluster. The report must include:

- strong and weak scaling on 1, 2, 4, 8 nodes
- speedup with respect to the sequential baseline
- a breakdown of the total execution time considering the communication/redistribution time, and local join time

You are encouraged to discuss the trade-off between local computation and communication, and to explain how the distributed version compares with the shared-memory implementations from the previous modules.

Since the cluster has limited resources, the required experimental campaign should focus on a small number of representative configurations. Extensive parameter sweeps are not required. The emphasis of this module is on selecting meaningful experiments and discussing them clearly.

All experiments must clearly state the number of nodes and MPI processes used. If you also implement the hybrid MPI+OpenMP version, clearly report the rank/thread configuration used.

Notes

You are expected to make reasonable engineering choices and justify them. There is no single correct MPI strategy, provided that correctness is preserved, the overall algorithmic structure remains recognizable, and the adopted choices are justified through analysis and experiments.

A central goal of this module is to understand that distributed performance depends not only on local computation, but also on communication costs and on the balance between them.

Deliverables

Send the teacher an email with subject “SPM Modulo4” and attach a zip file named Modulo4_NameSurname.zip by **3 June** (soft deadline). The zip file must contain:

- source code and build instructions (Makefile or explicit compile commands)
- a README containing instructions on how to run the tests and perform correctness checks
- a short PDF report (max 6 pages) that includes:
 - the MPI strategy adopted
 - strong and weak scaling results
 - performance breakdowns
 - a brief discussion of the results obtained and comparison with the previous shared-memory versions
 - if implemented, a separate discussion of the MPI+OpenMP version (up to 1 extra page)